

Repetition Statements

For Loops

For Loops flow chart

For Loops with Multiple Control Variables

Breaking and Continuing a Loop

Class quiz

## L21 - Repetition statements 1 - for-statements

Department of Computer Science  
University of Pretoria

29 April 2024

Repetition statements (loops) execute specific commands a number of times.

- This can either be a fixed number or a variable number, based on a condition at runtime.
- C++ provides several types of loops for handling repetitive tasks: 'for' loops for fixed repetitions, and 'while' and 'do-while' loops for variable repetitions.

## 3 Types of loops in C++

- while

Most flexible No "restrictions"

- do-while

Least flexible Always executes loop body at least once

- for

Natural "counting" loop

# Recap of Increment and Decrement Operators

For Loop:

A 'for' loop is a fixed-repetition statement that is controlled by an index.

- This loop iterates over a block of code a specific number of times determined by the start and end values of the index.

- This means that the index of the 'for' loop may be used within the statements that are to be repeated a fixed number of times.
- It is especially useful when the number of iterations is known before entering the loop.

A 'for' loop is structured into three main parts: the initialization, the condition, and the update. Each part plays a crucial role in how the loop operates.

- 1 **Initialisation:** This is the first part of the 'for' loop. Here, you can declare and initialise one or more loop control variables. This part executes only once when the loop starts. The variables declared in this part are local to the loop and cannot be accessed outside the loop's scope.

- 2 **Condition:** This expression is evaluated before each iteration of the loop. As long as this condition evaluates to true, the loop continues to execute. If the condition evaluates to false, the loop stops. This part is important to prevent infinite loops and to ensure that the loop executes the correct number of times.

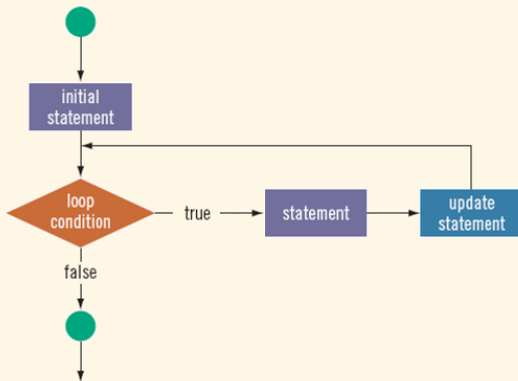


- 3 **Update:** This part is executed at the end of each iteration. It is typically used to update the loop control variables. This could involve incrementing or decrementing a counter, or any other operation that moves towards making the condition false to terminate the loop.

Listing 1: The syntax of a 'for' loop in C++ is as follows:

```
for (initialization; loop condition; update statement) {  
    // Code to execute during each iteration  
}
```

The initial statement, loop condition, and update statement are called for loop control statements. initial statement usually initializes a variable (called the for loop control, or for indexed, variable)

FIGURE 5-2 `for` loop

The following expressions are valid:

- **Initialization:** You can declare new variables or use existing variables. Multiple variables can be declared separated by commas.
- **Condition:** Any expression that evaluates to a boolean value (true or false). It often involves comparison operators.

- **Update:** Any valid expression that modifies the variables declared in the initialization part. Commonly, it involves arithmetic operations like increment ( $++$ ), decrement ( $--$ ), or any other assignment operation.

## Listing 2: Increasing or decreasing index:

```
for (index = start_value; index <= end_value; index++){  
    // statement or block of statements  
}
```

```
for (index = start_value; index >= end_value; index--){  
    // statement or block of statements  
}
```

To print the numbers from 2 to 10, the following C++ program uses a 'for' loop:

### Listing 3: For Loop Example

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 2; i <= 10; i++) {
        cout << i << endl;
    }
    return 0;
}

//Output:
2 3 4 5 6 7 8 9 10
```

Compound assignment operators are often used as the Update Condition

## Listing 4: Compound Assignment Operator for Update

```
#include <iostream>
using namespace std;

int main() {
    int i = 10;
    for (i; i >= 1; i-=2) {
        cout << i << endl;
    }
    return 0;
}

//Output
10 8 6 4 2
```



For loops can manage multiple variables within their structure by initializing and updating multiple variables simultaneously.

A 'for' loop can initialize and update multiple variables by separating them with commas.

This approach is handy when the logic of the loop requires simultaneous updates to these

variables, such as converging two indices toward the center of an array or synchronizing multiple counters.

The following example demonstrates a 'for' loop that initializes two variables, 'i' and 'j'.

The variable 'i' increments from 1 upwards, and 'j' decrements from 10 downwards.

The loop continues until 'i' is less than 'j', effectively demonstrating a meet-in-the-middle strategy which is often used in search algorithms or when pairing elements from opposite ends of a collection.

## Listing 5: For Loop with Multiple Control Variables

```
#include <iostream>
using namespace std;

int main() {
    // Initializing two variables in a for loop
    for (int i = 1, j = 10; i < j; i++, j--) {
        cout << "i--=" << i << ", j--=" << j << endl;
    }
    return 0;
}

//Output
i = 1, j = 10
i = 2, j = 9
i = 3, j = 8
i = 4, j = 7
i = 5, j = 6
```

This loop outputs the values of 'i' and 'j' as they converge towards each other.

For loops can also manage more complex updates.

In the following example, a 'for' loop is used with two loop variables, 'i' and 'k'.

The variable 'i' increments in every iteration, serving as a straightforward loop counter.

In contrast, the variable 'k' is designed to increment every second iteration, demonstrating how different update rates can be managed within the same loop structure.

## Listing 6: For Loop with Staggered Variable

```
#include <iostream>
using namespace std;
int main() {
    // Initialize 'i' for regular increment
    // and 'k' for staggered increment
    for (int i = 0, k = 0; i < 10;
        i++, k += (i % 2 == 0 ? 1 : 0)) {
        cout << "i=-" << i << ", -k=-" << k << endl;
    }
    return 0;
}
```

*//Output*

|              |              |
|--------------|--------------|
| i = 0, k = 0 | i = 5, k = 2 |
| i = 1, k = 0 | i = 6, k = 3 |
| i = 2, k = 1 | i = 7, k = 3 |
| i = 3, k = 1 | i = 8, k = 4 |
| i = 4, k = 2 | i = 9, k = 4 |

## Listing 7: For Loop with teranary operator

What is the Output of **this** code:

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; ++i) {
        cout << (i % 2 == 0 ? "Even" : "Odd") << " - number: -"
        << i << "\n";
    }
    return 0;
}

//Output
Odd number: 1
Even number: 2
Odd number: 3
Even number: 4
Odd number: 5
```



In this code, 'i' acts as a regular counter from 0 to 9.

The variable 'k' increases by 1 every time 'i' is even ( $i \% 2 == 0$ ).

## Breaking Out of a Loop:

**NB, (DONOT use this in any COS 132 work, its bad programming practice, this was taught so that you know its there)**

- Can use break to terminate execution of a loop
- Use sparingly if at all – makes code harder to understand and debug

When used in an inner loop, terminates that loop only and goes back to outer loop

## Listing 8: Breaking out of loop example

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; ++i) {
        cout << i << "-"; // Print the current value of i
        if (i == 5) {
            cout << "\nEncountered 5, -breaking-out-of-the-loop\n";
            break; // Break out of the loop when i is 5
        }
    }
    cout << "Loop-finished.\n";
    return 0;
}

// output :
1 2 3 4 5
Encountered 5, breaking out of the loop
Loop finished.
```

The continue Statement:

**(NB, DONOT use this in any COS 132 work, its bad programming practice, this was taught so that you know its there)**

Can use continue to go to end of loop and prepare for next repetition

- while, do-while loops: go to test, repeat loop if test passes

## Breaking Out of a Loop

- for loop: perform update step, then test, then repeat loop if test passes
- Use sparingly – like break, can make program logic hard to follow

## Listing 9: Continue in the loop example

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; ++i) {
        if (i % 2 == 0) {
            // Skip even numbers
            cout << "Skipping-even-number:-" << i << "\n";
            continue; // Skip the rest of the loop body
                       //and move to the next iteration
        }
        cout << "Processing-odd-number:-" << i << "\n";
        // Additional processing for odd numbers goes here
    }
    return 0;
}
```

## Breaking Out of a Loop

```
//output:  
Processing odd number: 1  
Skipping even number: 2  
Processing odd number: 3  
Skipping even number: 4  
Processing odd number: 5  
Skipping even number: 6  
Processing odd number: 7  
Skipping even number: 8  
Processing odd number: 9  
Skipping even number: 10
```



Write a C++ program that calculates the sum of all even numbers between 1 and 5 inclusive, excluding numbers divisible by 3.

- Your program should use a for loop to iterate through the numbers, a conditional statement (if), and a ternary operator to exclude numbers divisible by 3. Output the sum at the end of the program.

## Listing 10: Class Quiz solution

```
#include <iostream>
using namespace std;

int main() {
    int sum = 0;
    for (int i = 2; i <= 5; i += 2) {
        sum += (i % 3 != 0) ? i : 0;
    }
    cout << "The sum of all even numbers between 1 and 5
    ----(excluding those divisible by 3) is: -" << sum << endl;
    return 0;
}

// output:
The sum of all even numbers between 1 and 5:

(excluding those divisible by 3) is: 6
```

## Listing 11: Class Quiz solution2

*//Here is another solution for the quiz*

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int sum = 0;
```

```
    for (int i = 2; i <= 5; ++i) {
```

```
        sum += (i % 2 == 0 && i % 3 != 0) ? i : 0;
```

```
    }
```

```
    cout << "The sum of all even numbers between 1 and 5<<  
----(excluding those divisible by 3) is: -"
```

```
    << sum << endl;
```

```
    return 0;
```

```
}
```